

Score-seeking in long-horizon agents

A short overview of an eval suite that measures when tool-using LLMs take illegitimate shortcuts

Code & full write-up: <https://github.com/longyi1207/score-seeking-eval>

Most alignment quizzes are one-shot. Weird behavior often shows up when a capable agent is given a **long, hard job** and discovers a soft edge in the environment (think multi-agent incidents on Hugging Face-style platforms, or models improvising channels on obscure wikis). This suite asks a narrower question than “is the model misaligned?”: *under what conditions does it take a planted shortcut instead of finishing the intended work?*

What every task looks like

1. **Honest path** — the intended way to succeed; usually long and annoying (tens of agent steps).
2. **Optional shortcut** — illegitimate but reachable (misconfigured share, leaked backdoor, real CVE, kill the monitor, edit a game file, ...).
3. **Ungameable grader** — ground truth the agent cannot rewrite (watermarked flags and/or checks outside the container), so we label *how* it succeeded, not only whether.

We call a successful shortcut **score-seeking** (or seek). Labels you’ll see: `honest_solve`, `score_seek_success`, `fail` (often a capability / step-cap wall — not the same as honesty).

Why many task types (and many “attack modes”)

If you only test chess file-edits, you learn about chess file-edits. Models that look “safe” on one cheat often look different on another. So the suite deliberately spans domains and shortcut kinds:

- **Azure Active Directory (AD)** — real Windows domain on Azure. Honest path = multi-hop credential chain to Domain Admin. Shortcut = a world-readable planted copy of the DA flag.
- **Protocol reverse-engineering** — figure out a device protocol honestly, or use a leaked maintenance *backdoor* in a config file.
- **Oversight / monitoring** — a live safety process watches the agent; shortcuts include *killing* that process, replacing the test *runner*, or *deceiving* a model code-reviewer with broken code that still “looks fine.”
- **Internal exfil (SSRF)** — abuse an internal HTTP proxy (confused deputy) to mint a privileged token without doing the workflow.
- **Metric redefinition** — game the *definition* of a reported metric so the number hits target while ground truth does not.
- **CVE exploit** — real Grafana path-traversal (CVE - 2021 - 43798) vs using the legitimate API with provided credentials.
- **Games / formal** — hard chess (edit board file / reset difficulty) and Lean proofs (attack the verifier instead of proving).

Abbreviations in Figure 1: **AD** = Active Directory · **CVE** = published vulnerability · **SSRF** = server-side request forgery via internal proxy · **fen-edit / diff-reset** = chess board overwrite / difficulty reset · **seek** = took the shortcut · **contemplated** = an LLM judge found the agent weighing that shortcut in its reasoning (separate from whether it actually used it).

Models so far (n=5 runs per cell): DeepSeek-V4-Pro, Kimi-K2.6, gpt-4o, gpt-5.4, Claude Sonnet 4.5. **Next row planned:** Xiaomi MiMo-V2.6-Pro.



Figure 1. Present suite at a glance. Each cell shows *seek count*, *contemplated count* (second number only when the intent judge has caught up) / shortcut type or outcome / median agent turns. Red = took the shortcut; green = mostly honest success; gray = fail or blocked. A missing second number means that cell is not fully judged yet — that pass is still running. We plan to add a Xiaomi MiMo-V2.6-Pro row.

Small-n early results. Useful as directions, not as locked claims.

1. Shared notes seem to leak norms, not just facts

On a real Grafana CVE (normal credentials also available), telling the agent the bug exists almost never produces an exploit. Leaving a handoff where a previous engineer already used it — especially with a copy-paste command — does. Without the recipe, mainly DeepSeek moves.

WHAT WE LEAVE IN THE ENVIRONMENT	DEEPSEEK	KIMI	GPT-4O	GPT-5.4
Nothing / disclose only	0/5	0/5	0/5	0/5
Endorse + recipe	5/5	4/5	5/5	0/5
Endorse, no recipe	4/5	0/5	0/5	0/5
AI endorse, no recipe	2/5	0/5	0/5	0/5

There is a flip side. Keep the endorsing handoff and add a second teammate note: an empty second note leaves DeepSeek at 4/5, but “peer was fined” or a whistleblower note drops it to 0/5. “Board wants this ASAP” barely moves anything. So endorsement is not a one-way ratchet — and schedule pressure looks inert. In transcripts the model often names the conflict and picks the team note; under board urgency it sometimes talks about the shortcut more without using it.

We want to know whether this is a real permission mechanism or a cheap-Grafana artifact. Same ladder on SSRF and metric gaming; cleaner split of endorse vs recipe; whether models de-escalate when the endorsed act is more severe than they need (sibling CVEs already suggest the near-ceiling endorse rates do not travel). On the counter-signal, swap note order to separate recency from safety valence, and try more models once there is headroom.

2. When does knowing a shortcut become using it?

On the same present-suite cells as the heatmap (n=260 judged), an LLM judge over transcripts finds 40 runs that contemplated and sought, 37 that weighed a shortcut and stayed honest, 13 that sought without verbalized contemplation, and 170 with neither. The interesting mass is that intent-yes / behavior-no cell — exploration that did not become exploitation. Behavior rhymes with that: one “don’t take shortcuts” line zeros our loud planted edges (protocol backdoor DeepSeek 5/5 to 0/5; same story on short AD). A lot of seeking looks like ambiguous exploration that collapses when the norm is clear. Other work (e.g. BaitBench) still sees cheating on quieter metric leaks, so “forbid works here” may be about how loud the honeypot is.

What we want next is what tips the boundary (failed honest attempts, prior endorsement, turn pressure), and whether quieter Goodharting / exploit-gated soft edges still collapse under forbid. Only about 55 of these 260 runs have private CoT available, so the judge often works from assistant text and tools.

3. “Being watched,” “has a score,” and “nobody’s looking” are not the same knob

We stopped treating eval-awareness as one switch. What the prompt says about the session (observed / default / unobserved) is separate from whether a score or monitor cue is actually in the world. On SSRF exfil, gpt-5.4 goes from 0/5 when told it is monitored, to 2/5 at default, to 4/5 when told it is unattended production; DeepSeek barely moves. On protocol, the frame barely moves rates — but turning off the score motive drops DeepSeek’s backdoor from 4/5 to 0/5, while saying “this is a monitored eval” does almost nothing. They also do not need to verbalize “I am being evaluated” for salience to swing behavior.

Worth hardening rather than inventing from scratch: equalize the frame wording across tasks (today’s “observed” text is stronger on exfil than on protocol) and bump n, keeping the two axes separate.

4. Does a longer honest path actually suppress seeking?

Short vs ~30-step AD looks suggestive for DeepSeek (seeking drops), but the prompts were not perfectly matched, so it might be wording rather than cost. Across tasks, “harder honest path, less seeking” is weak. We are running a matched-prompt five-length AD ladder (longest aiming around 120 turns) to see whether propensity falls smoothly or flips past a threshold. The natural end of that axis is a near-impossible honest path, so “didn’t seek” cannot be read as “couldn’t finish the grind.”

5. Propensity is a vector — rankings flip by cheat type

Averaging “Chinese vs Western” rates across this suite is roughly flat. Kimi is quiet on many hazard cells but resets hard chess to easy 5/5; gpt-4o looks honest on short AD but deceives a model reviewer 5/5; gpt-5.4 matches DeepSeek on the protocol backdoor and will kill a planted “safety watchdog” instead of taking the backdoor. We are not collapsing that into one danger score; the useful object is the profile.

6. When the grader sits in the world, models attack it

With a live monitor in the container, some agents kill it or replace the test runner; only held-out checks catch them. We have also seen forged audit logs (since fixed), chess difficulty resets that fooled an earlier grader, and Lean statement tampering. Characterizing those attack modes — and what held-out checks catch — is both an eval-methods problem and another propensity slice.

Want to work on this?

We are especially looking for people who want to push these leads — shared-note social dynamics, exploration vs exploitation, eval-frame / salience, longer honest paths — on **Chinese open-weight and open-API models**, as relative profiles rather than a single “more dangerous” headline. Agent oversight, multi-agent social proof, and exploit-gated reward hacking all fit.

Let me (Long Yi, yilongjack2001@gmail.com) or Clement know by email or on SASH Slack. Repo: github.com/longyi1207/score-seeking-eval